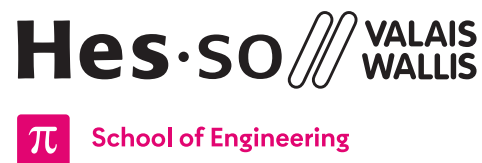




Game of Life (GoL)

Cours Architecture de l'ordinateur (CAr)



Orientation: Informatique et systèmes de communication (ISC)

Spécialisation: Data Engineering (DE)

Cours: Architecture de l'ordinateur (CAr)

Auteurs: Silvan Zahno, Axel Amand, Rémy Borgeat

Date: 05.08.2026

Version: v1.0



Contenu

1	Introduction	3
2	Spécifications	4
2.1	Spécification minimale	6
2.2	Règles	6
2.2.1	Grille toroidale	8
2.2.2	Conseils	8
3	Software	9
3.1	Grilles et représentation des cellules	9
3.1.1	Grille	9
3.1.2	Cellule	9
3.2	Tâches	10
3.2.1	gol_init	10
3.2.2	gol_step	10
3.3	Compiler et flasher	11
3.4	Tests	12
3.4.1	Hardware In The Loop (HITL)	12
3.5	Utilisation du GoL Viewer	13
3.5.1	Configuration	13
3.5.2	Visualisation de la grille	14
3.5.3	Édition de la grille	14
4	Evaluation	16
	Annexes	17
1	Debug (Optional)	17
1.1	Mise en place	17
1.1.1	Extensions	17
1.1.2	Pilotes	17
1.1.3	Debug	17
	Glossaire	21



1 Introduction

Le but du projet est d'appliquer les connaissances acquises à la fin du semestre directement à l'aide d'un exemple pratique.

L'objectif est de mettre en œuvre le [Conway's Game of Life \(GoL\)](#), un système d'automates cellulaires basé sur 4 règles simples, agissant « par lui-même » - *également connu comme jeu à zéro joueur* -.

Le [Jeu de la vie](#) a été créé par le mathématicien britannique John Conway en 1970.

Une grille est composée de cellules, chacune pouvant être vivante ou morte. L'évolution est basée sur 4 règles simples appliquées à chaque cellule en fonction de ses voisines - voir [Chapitre 2.2](#).

Le jeu utilise une application de bureau pour créer une grille de cellules, puis l'envoie à une [ESP-RS devboard](#).

Le microcontrôleur applique les règles du jeu à la grille et renvoie le résultat à l'application pour visualiser l'évolution du système en direct. La [Light Emitting Diode \(LED\)](#) rouge sur la carte bascule chaque fois que la grille est mise à jour pour montrer qu'elle est en fonctionnement. Ce système est représenté dans l'illustration [Figure 1](#):

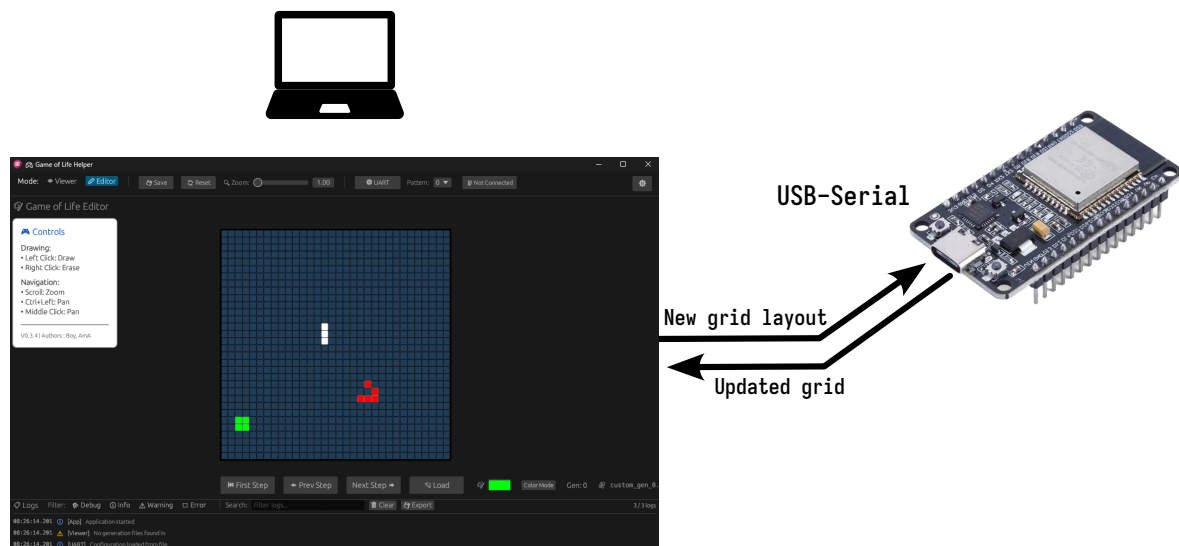


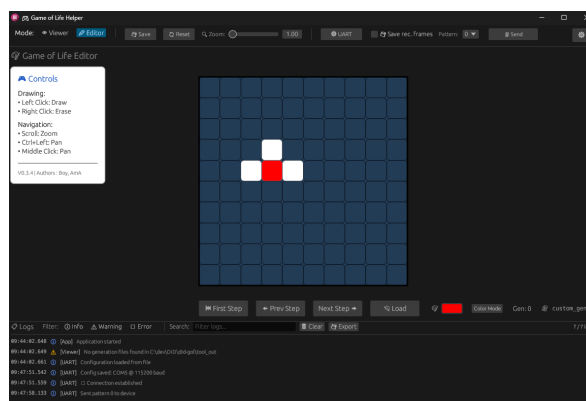
Figure 1 - Équipement du GoL

L'objectif est d'implémenter la génération de la grille en langage assembleur conformément à la spécification [Chapitre 2](#).



2 | Spécifications

Le système utilise le **GoL Viewer**, une application de bureau, pour créer et visualiser des grilles de cellules



Les cellules sont ensuite envoyées à une [ESP-RS devboard](#).

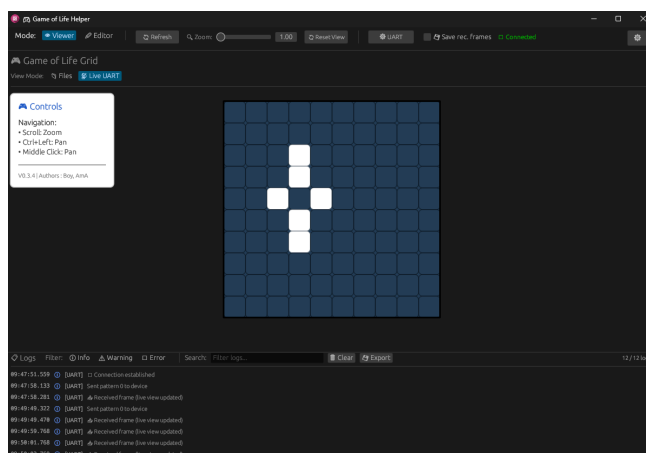
Cette carte embarque une puce [ESP32-C3](#) hébergeant un microcontrôleur [5-stages Reduced Instruction Set Computer architecture, open-sourced \(RISCV\)](#) monocœur 32 bits prenant en charge les ensembles d'instructions **RV32IMC**.

Il exécute une application basée sur [Embedded Rust](#), utilisant Rust et partiellement du code assembleur [RISCV](#) à des points clés, comme le montre le diagramme [Figure 4](#).

Le code assembleur est divisé en **deux parties**:

1. **gol_init**: configure un motif de grille par défaut par le microcontrôleur
2. **gol_step**: reçoit une grille du **GoL Viewer** et doit appliquer les règles du **GOL** dessus, puis l'envoyer au visualiseur

Lorsque des grilles sont reçues, l'application **GoL Viewer** les affiche à l'écran:



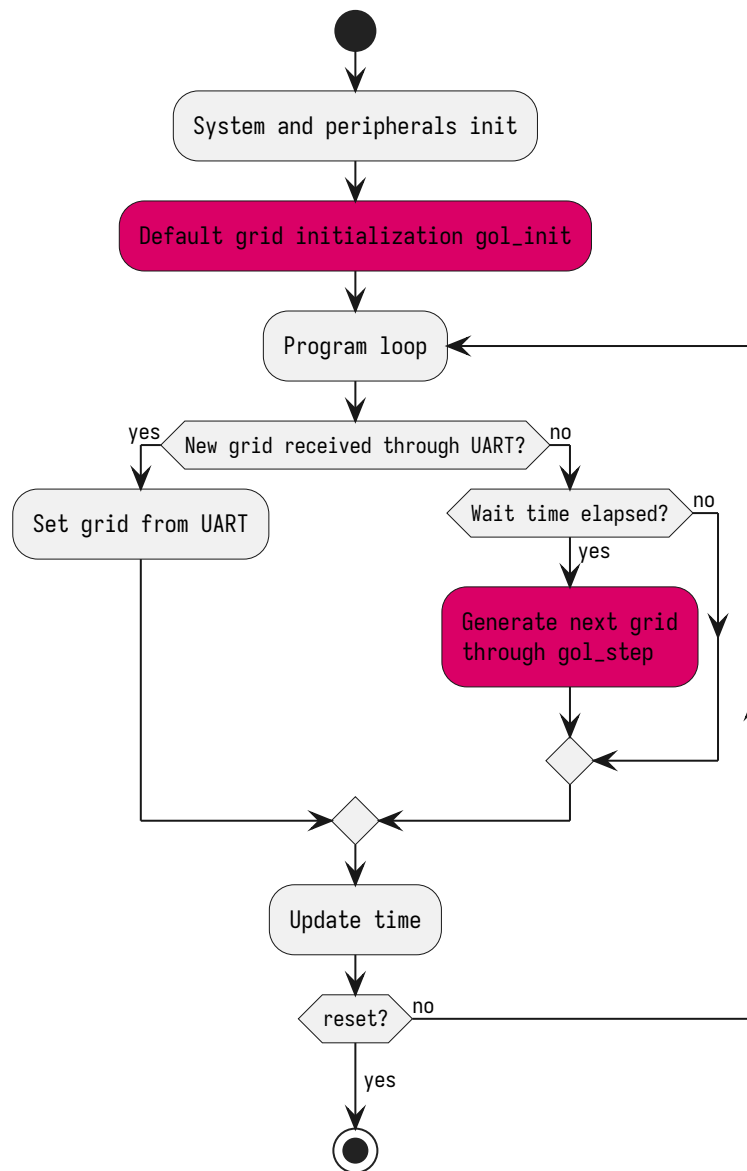


Figure 4 - Diagramme de flux du système pour l'implémentation du Game of Life



L'extension **C** du **RV32IMC** jeu d'instructions définit des instructions compressées, particulièrement utiles sur les systèmes embarqués avec une mémoire limitée. Dans ce projet, cette **Instruction Set Architecture (ISA)** peut être ignorée lors de l'écriture de code assembleur direct.



2.1 Spécification minimale

Le système doit implémenter les fonctionnalités suivantes:

- Au démarrage dans **gol_init**, le système doit afficher un motif de grille par défaut à l'écran sans aucune interaction de l'utilisateur depuis le **GoL Viewer**. La grille doit comporter 32x32 cellules et contenir un *blinker* - [Figure 5](#) - au centre de la grille.
- À chaque appel de **gol_step**, la grille doit être mise à jour selon les règles du **GoL** [Chapitre 2.2](#) et renvoyée à l'application **GoL Viewer** pour affichage. Les règles doivent être appliquées de manière *toroïdale*, ce qui signifie que la grille se replie sur ses bords - voir [Chapitre 2.2.1](#).
- Le code doit suivre les conventions et utiliser des appels de fonctions pour éviter la duplication de code et améliorer la lisibilité et le débogage. Le code doit être commenté pour expliquer les fonctions et la logique derrière l'implémentation.
- Un rapport doit être rédigé conformément au [Chapitre 4](#).
- Un résumé graphique du flux de code doit être fourni, similaire à la [Figure 2](#), montrant les principales étapes du code de **gol_init** comprenant les décisions et les appels de fonctions.
- Un résumé graphique du flux de code doit être fourni, similaire à la [Figure 2](#), montrant les principales étapes du code de **gol_step** comprenant les décisions et les appels de fonctions.
- Aucun contenu généré par l'IA n'est autorisé dans le code ou le rapport. Le code et le rapport doivent être rédigés par l'étudiant.

2.2 Règles

GoL est un système d'automates cellulaires qui consiste en une grille de cellules, chacune étant *vivante* ou *morte*. L'évolution de la grille suit quatre règles simples basées sur la cellule actuelle et ses voisines:

1. Sous-population

Toute cellule vivante avec moins (< 2) de deux voisines *meurt*



2. Survie

Toute cellule vivante avec deux ($= 2$) voisines vivantes *survit* à la génération suivante



3. Surpopulation

Toute cellule vivante avec plus de trois (> 3) voisines vivantes *meurt*



4. Reproduction

Toute cellule morte avec exactement trois ($= 3$) voisines vivantes devient *vivante*



Ces règles simples créent des comportements émergents fascinants, notamment des motifs stables, des oscillateurs et des structures mobiles appelées « gliders », comme le montre la [Figure 5](#):

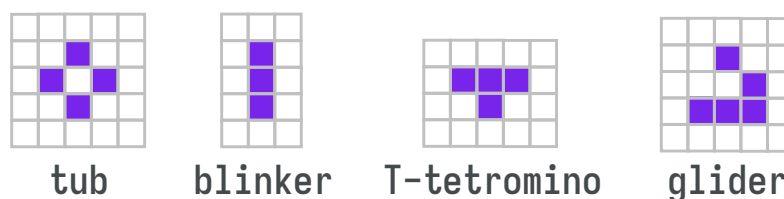


Figure 5 - Motifs intéressants dans **GoL** : structures stables, oscillateurs et gliders



Le jeu peut être expérimenté sur <https://playgameoflife.com> ou <https://lifeee.netlify.app>.



2.2.1 Grille toroidale

Une grille **toroïdale** est une grille où les bords s'enroulent, créant une surface « infinie » sans limites. Appliquée à ce projet, cela signifie que certains voisins des cellules aux extrémités de la grille apparaissent du côté opposé.

Par exemple, prenons la grille 6x6 suivante et vérifions les voisins de la cellule (0,0):

0,0	1,0				5,0
0,1	1,1				5,1
0,5	1,5				5,5

1. Les voisins de gauche débordent vers la droite de la grille
2. Les voisins du haut débordent vers le bas de la grille
3. Le voisin en haut à gauche débordé vers le bas à droite de la grille

Figure 6 - Voisins de la cellule (0, 0)

En généralisant cela, les voisins d'une cellule à la position (x, y) dans une grille de taille (X_Size, Y_Size) sont:

(22,16)	(23,16)	(24,16)
(22,17)	(23,17)	(24,17)
(22,18)	(23,18)	(24,18)

Figure 7 - Voisins de la cellule (23, 17)

grid_width-1, grid_height-1	0, grid_height-1	1, grid_height-1
grid_width-1, 0	0, 0	1, 0
grid_width-1, 1	0, 1	1, 1

Figure 8 - Voisins de la cellule (0, 0)

(x-1 + MAX_X) % MAX_X, (y-1 + MAX_Y) % MAX_Y	x % MAX_X, y % MAX_Y	(x+1) % MAX_X, (y-1 + MAX_Y) % MAX_Y
(x-1 + MAX_X) % MAX_X, y % MAX_Y	x % MAX_X, y	(x+1) % MAX_X, y % MAX_Y
(x-1 + MAX_X) % MAX_X, (y+1) % MAX_Y	x % MAX_X, (y+1) % MAX_Y	(x+1) % MAX_X, (y+1) % MAX_Y

Figure 9 - Voisins d'une cellule générique (x, y)

2.2.2 Conseils

1. Commencez par tester les jeux [GoL](#) sur le web pour comprendre les règles et leur fonctionnement.
2. Commencez par définir le flux de code, les fonctions, les décisions ... et dessinez un diagramme similaire à [Figure 2](#). Cela vous aidera à comprendre le système et comment l'implémenter.
3. Implémentez la fonction `gol_init` pour vous familiariser avec les lectures et écritures en mémoire. Remplacez la ligne `unsafe { gol_step(cur, nxt, COLS as u32, ROWS as u32) }` dans `main.rs` par `rust_send_grid(cur as *const u32, CELLS as u32)`; pour afficher et visualiser cette grille dans le GoL Viewer.
4. Restaurez le code rust `unsafe { gol_step...` et implémentez la fonction `gol_step` pour appliquer les règles du jeu à la grille. Allez-y par petites étapes et testez souvent en exécutant votre code et en visualisant l'évolution dans le GoL Viewer. Si nécessaire, itérez petit à petit avec le débogueur - voir [Chapitre 1](#).
5. Une fois que tout est en place, exécutez les tests pour vous assurer que la génération et les règles toroïdales sont correctement implémentées - voir [Chapitre 3.4](#).



3 | Software



Le code [Assembly \(ASM\)](#) à implémenter se trouve dans `src/asm/gol.s` et est appelé par le code Rust dans `src/bin/main.rs`.

3.1 Grilles et représentation des cellules

Comme le code assembleur est mélangé avec du code Rust donné, il est important de comprendre certaines notions importantes au préalable.

3.1.1 Grille

Les grilles sont définies dans `grid.rs` comme suit:

```
pub static mut GRID_X: [u32; WIDTH * HEIGHT] = [0u32; WIDTH * HEIGHT];
```

En mémoire, une grille est stockée comme un tableau 1D contigu de **Largeur** * **Hauteur** cellules, mais peut être vue comme un tableau 2D où chaque cellule est donnée par ses coordonnées (**x**, **y**). Prenons pour exemple une grille de 3x3 cellules:

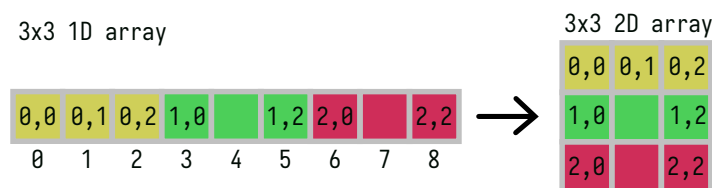


Figure 10 - Grille en mémoire



En interne, deux grilles **A** et **B** sont utilisées, notamment pour le [Chapitre 3.2.2](#). Cela permet de calculer la prochaine génération de la grille dans **B** tandis que la génération actuelle est lue dans **A** sans perdre de données. Une fois le calcul terminé, les deux grilles sont échangées.

3.1.2 Cellule

Comme on peut le voir dans l'[extrait de code précédent 1](#), la grille est définie comme un tableau d'entiers non signés de 32 bits.

Chaque cellule est en réalité représentée par une valeur 32bit `xxRRGGBB`, où `xx` est un octet réservé (fixé à `0x00`, et `RRGGBB` est la couleur de la cellule au format hexadécimal.

Cette couleur est également gérée par le **GoL Viewer** et peut être utilisée pour des fonctionnalités avancées comme la mise en œuvre d'un [Jeu de la vie arc-en-ciel](#) où chaque cellule hérite des couleurs lorsqu'elle naît.

Une cellule **morte** est déterminée par le fait que tous ses bits **Red Green Blue (RGB)** sont à 0, tandis que toutes les autres valeurs de **RGB** sont considérées comme des cellules **vivantes**:

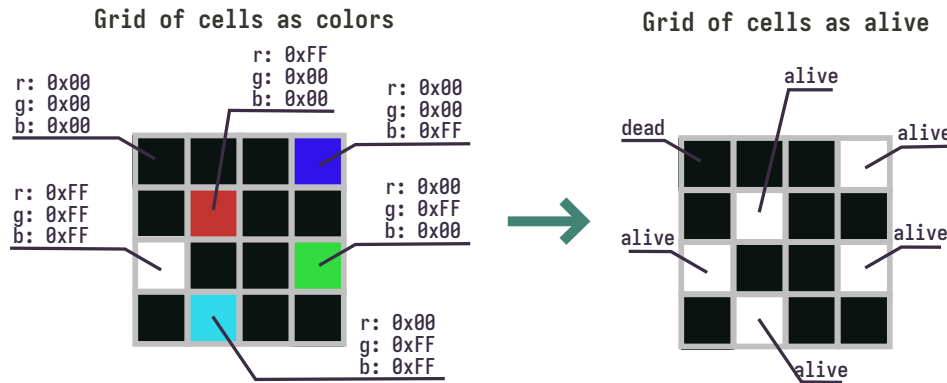


Figure 11 - Statut d'une cellule vivante ou morte

3.2 Tâches

3.2.1 gol_init

La fonction **gol_init** est appelée au démarrage et doit initialiser la grille avec le motif par défaut - un *blinker* au centre de la grille - Figure 5.

Elle est définie comme **fn gol_init(data_ptr: *mut u32, cols: u32, rows: u32)**; où:

- **data_ptr** contient l'adresse mémoire de la grille, cellule (0, 0). Faites attention à n'accéder qu'aux emplacements mémoire autorisés car il n'y a pas de vérification des limites.
- **cols** est le nombre total de colonnes dans la grille
- **rows** est le nombre total de lignes dans la grille



1. Dessinez un diagramme de flux représentant les étapes pour initialiser la grille avec un motif de blinker au centre.
2. Itérez sur toute la grille pour décider si une cellule doit être définie comme **morte** (0xXX000000) ou **vivante** (toute autre valeur selon que les couleurs sont utilisées ou non).
3. Initialiser le motif de la grille au démarrage en plaçant un blinker - Figure 5 - en son centre.

3.2.2 gol_step

La fonction **gol_step** est appelée chaque fois que la prochaine génération de la grille doit être calculée.

Elle est définie comme **fn gol_step(in_data_ptr: *mut u32, out_data_ptr: *mut u32, cols: u32, rows: u32)**; où:

- **in_data_ptr** contient l'adresse mémoire de la grille d'entrée, cellule (0, 0). Faites attention à n'accéder qu'aux emplacements mémoire autorisés car il n'y a pas de vérification des limites. Ne lisez que depuis celui-ci.
- **out_data_ptr** contient l'adresse mémoire de la grille de sortie, cellule (0, 0). Faites attention à n'accéder qu'aux emplacements mémoire autorisés car il n'y a pas de vérification des limites. N'écrivez que dans celui-ci.
- **cols** est le nombre total de colonnes dans la grille
- **rows** est le nombre total de lignes dans la grille



La recette pour calculer la prochaine génération est la suivante:

1. Itérez sur toutes les cellules d'entrée. Pour chacune:
 - Itérez sur tous les voisins de la cellule actuelle, en comptant combien sont **vivants**
 - Appliquez les règles du [Chapitre 2.2](#) pour déterminer si la cellule actuelle est vivante ou morte dans la prochaine génération
 - Enregistrez la nouvelle valeur dans la grille de sortie aux mêmes coordonnées que la cellule actuelle
2. Une fois la grille entièrement calculée, envoyez-la via **Universal Asynchronous Receiver Transmitter (UART)** en appelant la fonction d'aide `fn rust_send_grid(ptr: *const u32, count: u32);` où:
 - **ptr** contient l'adresse mémoire de la grille de sortie, cellule **(0, 0)**
 - **count** est le nombre total de cellules dans la grille, c'est-à-dire **Largeur * Hauteur**



Pour appeler cette fonction, respectez la convention d'appel [RISCV](#). Vous pouvez ensuite effectuer un appel au label `rust_send_grid` pour accéder à la fonction.

3. Enfin, échangez les grilles **A** et **B** pour préparer le calcul de la prochaine génération en appelant la fonction d'aide `fn rust_swap_grids();`.



Pour appeler cette fonction, respectez la convention d'appel [RISCV](#). Vous pouvez ensuite effectuer un appel au label `rust_swap_grids` pour accéder à la fonction.



1. Dessinez un diagramme de flux représentant les étapes pour calculer la prochaine génération de la grille.
2. Calculez la prochaine grille et transmettez-la par [UART](#).

3.3 Compiler et flasher

Pour déployer le logiciel GoL, utilisez le **justfile** donné pour compiler et flasher le cœur [RISCV](#):

```
# One by one
just gol-flash # Compile and flash the GoL software to the RISCV core
just gol-viewer # Launch the GoL Viewer to visualize the grid

# Both together
just run # Compile and flash the GoL software to the RISCV core,
        # then launch the GoL Viewer
```



3.4 Tests

En raison des capacités de débogage limitées - voir [Chapitre 1](#), seuls les tests logiciels complets peuvent être automatisés - voir [Chapitre 3.4.1](#). Le reste du code peut être implémenté avec les outils vus lors des laboratoires ou en flashant le [Central Processing Unit \(CPU\)](#) et en vérifiant les résultats dans le visualiseur.

3.4.1 HITL

Les tests finaux sont effectués dans un environnement [HITL](#), où le cœur [RISCV](#) exécute le logiciel GoL tandis qu'une application PC dédiée est utilisée pour définir et obtenir des résultats via [UART](#) et évaluer la justesse du logiciel.

Pour exécuter ces tests, utilisez le **justfile** donné pour lancer l'environnement [HITL](#) :

```
just test <port> # Compile and flash the #gls("riscv") core then launch the HITL
application to test the GoL software
# Port on Windows should be COMx, on Unix /dev/ttyUSBx or /dev/ttyACMx.
# You can also set the DEF_PORT variable in the justfile to avoid specifying the port
each time.
```

Ces tests vérifient:

1. Lorsqu'il est fraîchement flashé, la grille doit déjà contenir un motif par défaut contenant un blinker au centre de la grille
2. Ensuite, il envoie des grilles connues et vérifie quelques générations:
 - Block (motif stable)
 - Beehive (motif stable)
 - Blinker (oscillateur de période 2)
 - Pulsar (oscillateur de période 3)
 - Glider (motif mobile)

Après les tests, des fichiers journaux sont générés à côté du **justfile** pour fournir des rapports détaillés des tests et de leurs résultats ainsi qu'un résumé des tests:

```
Total tests: 6
Passed: 6
Failed: 0
grid_init: PASS
block: PASS
beehive: PASS
blinker: PASS
pulsar: PASS
glider: PASS
```

```
HIL summary started: port=COM3,
baud=115200 (summary log:
<path>\<to>\test_summary.log)
grid_init log:
<path>\<to>\test_grid_init.log
grid_init PASS
pattern=block PASS
pattern=beehive PASS
pattern=blinker PASS
pattern=pulsar PASS
pattern=glider PASS
HIL result: PASS (5 passed, 0 failed)
```



3.5 Utilisation du GoL Viewer

Le GoL Viewer est une application simple pour définir et visualiser la grille et son évolution. Elle est écrite en **rust** et se connecte au **CPU** via **UART**. Il est disponible pour Windows, Linux et MacOS.

Le visualiseur est automatiquement lancé à partir des recettes **just run** ou **just gol-viewer**. Il peut également être lancé manuellement en exécutant l'exécutable **gol-viewer** dans le dossier **GoLViewer** du projet:

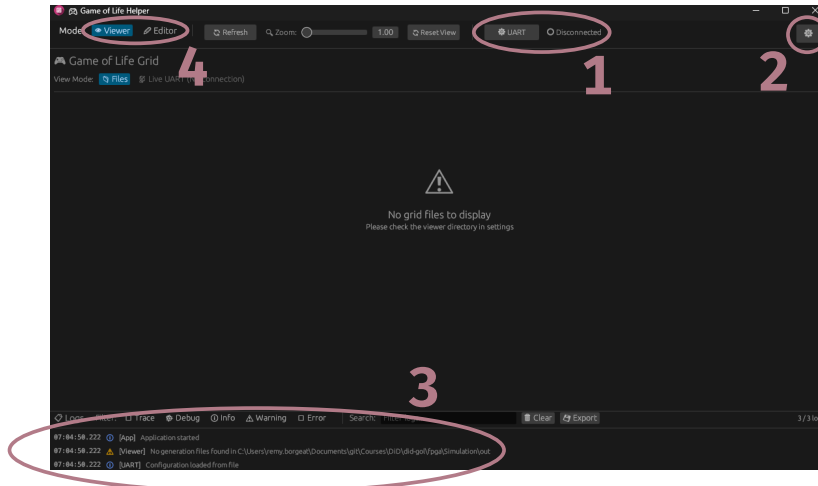


Figure 12 - GoL Viewer

1. Statut et configuration de la connexion **UART**
2. Paramètres du visualiseur et de la grille
3. Fenêtre de journalisation
4. Commutateur vue / édition

3.5.1 Configuration

Commencez par brancher la carte ESP, puis définissez le port **UART** correct. Sans configuration spéciale, le plus grand port de la liste devrait être le bon. Définissez le débit en bauds sur **115'200** et la configuration comme indiqué ci-dessous, puis cliquez sur **Connect**:

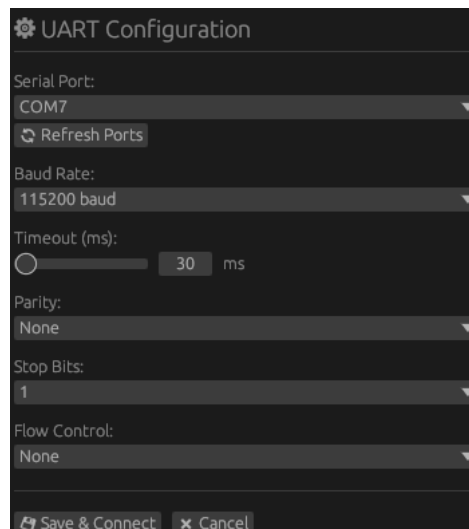


Figure 13 - Configuration **UART**



Ensuite, assurez-vous que le visualiseur est configuré sur la bonne taille de grille, qui est **32x32** par défaut.

Vous pouvez également définir un dossier d'entrée et de sortie pour lire et écrire les résultats à partir/vers des fichiers (optionnel).

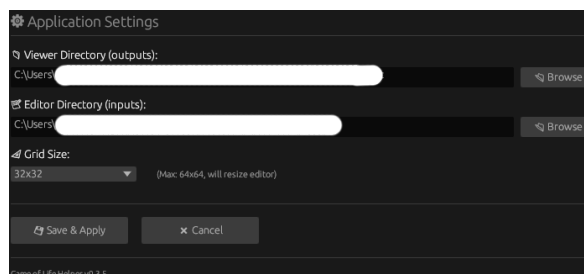


Figure 14 - Configuration de la grille

3.5.2 Visualisation de la grille

Passez en mode **Viewer** et sélectionnez l'option **Live UART** pour voir les grilles générées depuis la carte ESP:

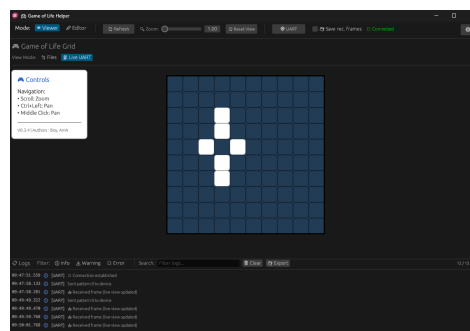


Figure 15 - Vue en direct

3.5.3 Édition de la grille

Passez en mode **Editor** pour créer une grille personnalisée.

Cliquez simplement avec le bouton gauche sur les cellules pour définir leur état sur vivant, cliquez avec le bouton droit pour les définir sur mort.

Cliquez sur le bouton **Classic Mode** pour activer le **Color Mode**. Sélectionnez une couleur et définissez les cellules comme d'habitude. Ces couleurs s'afficheront sur la première image et seront conservées lors de votre implémentation de **gol_step** [Chapitre 3.2.2](#).

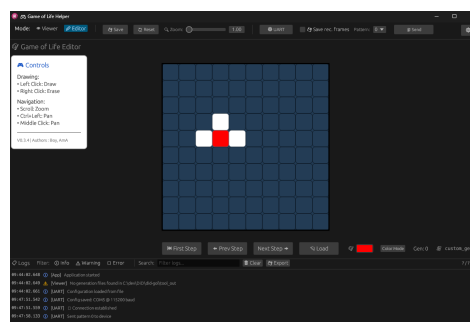


Figure 16 - Éditeur



Cliquez sur le bouton **Send** en haut à droite du visualiseur pour envoyer la grille. Revenez au mode **Viewer** [Chapitre 3.5.2](#) pour voir l'évolution de la grille.



4 | Evaluation

La note finale contient le rapport, le code ainsi qu'une présentation du système [Table 1](#).

Aspects évalués	Points
Rapport	50
Introduction	2
Spécification	2
Projet	10
Vérification et validation	10
Diagramme d'activité	20
Conclusion	2
Aspects formels du rapport	4
Fonctionnalité du circuit	20
Qualité de la solution	20
Présentation	20
Total	110

Table 1 - Grille d'évaluation



La grille d'évaluation donne des indications sur la structure du rapport. Pour un bon rapport, consultez le document **Comment rédiger un rapport de projet** [\[1\]](#).



Annexes

1 | Debug (Optional)

1.1 Mise en place

Bien que l'expérience ne soit pas sans inconvénients, il est possible de déboguer le logiciel GoL en parcourant le code assembleur tout en regardant les registres se mettre à jour. Pour ce faire, une configuration VSCode est déjà mise en place dans le projet, utilisant l'extension [Probe-rs Debugger](#).



Il est possible d'utiliser une combinaison d'autres éditeurs et visualiseurs en utilisant soit **probe-rs** soit **OpenOCD** pour déboguer le cœur [RISCV](#). Cependant, cela n'est pas couvert dans cette documentation.

1.1.1 Extensions

Installez l'extension [Probe-rs Debugger](#) dans VSCode pour activer le débogage du cœur [RISCV](#).

1.1.2 Pilotes

Les pilotes doivent être correctement configurés pour atteindre l'ESP32 via l'unité de débogage USB JTAG/série. Même si l'ESP32 communique correctement avec le PC hôte via [UART](#), le débogueur utilise le port **JTAG** pour interagir avec le cœur.

Pour les systèmes Unix, cela peut fonctionner directement. Si nécessaire, voir [Configuration de Probe-rs](#) pour plus d'informations.

Pour les utilisateurs Windows, il est nécessaire de modifier le pilote de l'ESP32 **USB JTAG/serial debug unit** via [Zadig](#). Le pilote **WinUSB v.6.1** est nécessaire à la place du pilote par défaut.

Ouvrez **Zadig**, sélectionnez **Options > List All Devices**, puis sélectionnez **USB JTAG/serial debug unit (Interface 2)**. Cliquez sur le menu déroulant près du bouton **> Install device** puis cliquez sur **Replace Driver**:

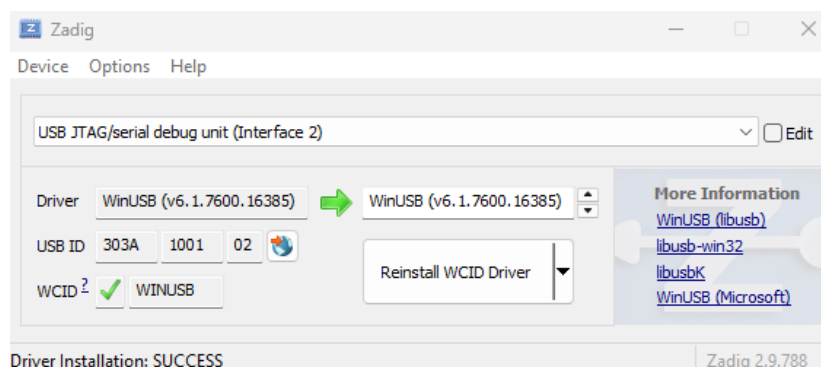


Figure 17 - Installation du pilote Zadig

1.1.3 Debug

Une fois tout mis en place, branchez la carte et:

- Ouvrez le visualiseur avec **just gol-viewer**



- Connectez le programme au bon port **UART** - cela permet d'imprimer des messages dans la console du visualiseur depuis le **CPU**

Référez-vous à l'image **Figure 19**:

1. Ouvrez le débogueur VSCode ① dans le panneau de gauche:
2. Sélectionnez la configuration **Debug GoL** et cliquez sur le bouton vert pour compiler et connecter le débogueur
3. Placez des points d'arrêt dans le code assembleur dans le fichier **GoLRS/src/bin/main.rs** devant la ligne **gol_step** en cliquant à gauche du numéro de ligne (même procédure pour **gol_init** si vous souhaitez déboguer l'initialisation de la grille)
 - Effacez les logs dans la console du visualiseur
4. Cliquez sur le bouton bleu  pour exécuter le code jusqu'à ce que le point d'arrêt soit atteint
 - Vérifiez les messages dans le visualiseur et notez les adresses données - elles correspondent à votre zone de code **ASM** dans le binaire compilé

```
11:04:33.745 [CPU] Initialising grid
11:04:33.745 [CPU] * cur_ptr = 0x3fc82e74, nxt_ptr = 0x3fc83e74
11:04:33.745 [CPU] * ASM region = 0x4201014c..0x42010494
11:04:33.745 [CPU] * gol_init = 0x42010168
11:04:33.745 [CPU] * gol_step = 0x42010390
11:04:33.745 [CPU] * Initialising grid with blinker oscillator in the centre
11:04:33.745 [CPU] Running GOL main loop
```

Figure 18 - Viewer log

5. Ouvrez le panneau de contrôle avec **Ctrl+Shift+P** et lancez **Open Disassembly View**



Cela ouvre un nouveau panneau avec du code assembleur. Il ne correspond pas encore à la fonction que vous avez écrite, vous devez encore itérer quelques étapes jusqu'à atteindre la prochaine instruction **jal**. Cliquez soit sur la flèche **Step Into**, soit sur **F11**:

0x420125A0	33 05 65 41	sub a0, a0, s6
539:	self.as_raw() >= 0	
0x420125A4	33 35 05 01	sltu a0, a0, a6
0x420125A8	01 4B	c.li s6, 0
0x420125AA	81 4B	c.li s7, 0
0x420125AC	23 2E A4 F5	sw s10, -0xa4(s0)
154:	if !frame_listener.frame_in_progress() && last_step.elapsed() >= step_period {	
0x420125B0	E3 12 05 A0	bnez a0, -0x5fc
156:	unsafe { gol_step(cur, nxt, COLS as u32, ROWS as u32) };	
0x420125B4	13 06 00 02	addi a2, zero, 0x20
0x420125B8	93 06 00 02	addi a3, zero, 0x20
0x420125BC	66 85	c.mv a0, s9
0x420125BE	A6 85	c.mv a1, s1
0x420125C0	97 E0 FF FF	auipc ra, 0xfffffe
0x420125C4	E7 80 00 DD	jalr ra, ra, -0x230
0x420125C8	37 45 00 60	lui a0, 0x60004
2107:	intrinsics::volatile_load(src)	
0x420125CC	48 41	c.lw a0, 4(a0)

Figure 20 - Disassembly view

En passant l'instruction **jal**, votre fonction devrait être atteinte. Le panneau de gauche montre les valeurs des registres:



VARIABLES	0x4201035C			
Static	0x42010360			
<&str as core::fmt::Debug>::(vtable)	0x42010364			
<riscv_pac::result::Error as core::fm	0x42010368			
<usize as core::fmt::Debug>::(vtable)	0x4201036C			
<&usize as core::fmt::Debug>::(vtable)	0x42010370			
<&str as core::fmt::Debug>::(vtable)	0x42010374			
<&str as core::fmt::Debug>::(vtable)	0x42010378			
riscv_rt	0x4201037C			
Registers	0x42010380			
x0/zero = 0x00000000	0x42010384			
x1/LR = 0x420125c8	0x42010388			
x2/SP = 0x3fcce2d0	0x4201038C	67 80 00 00		ret
x3/GP = 0x3fc8127c	0x42010390			
x4/TP = 0x00000000	0x42010394			
x5/T0 = 0x00ffffff	0x42010398			
x6/T1 = 0x00000000	0x4201039C			
x7/T2 = 0x00000003	0x420103A0			
x8/FP/S0 = 0x3fc82e74	0x420103A4			
x9/S1 = 0x3fc83e74	0x420103A8			
x10/A0/R0 = 0x3fc82e74	0x420103AC			
x11/A1/R1 = 0x3fc83e74	0x420103B0			
x12/A2 = 0x00000020	0x420103B4			
x13/A3 = 0x00000020	0x420103B8			
x14/A4 = 0x00000000	0x420103BC	93 84 05 00		mv s1, a1
x15/A5 = 0x00000000	0x420103C0			
x16/A6 = 0x000f4240	0x420103C4			
x17/A7 = 0x3fc80d0c	0x420103C8			
x18/S2 = 0x40000000	0x420103CC			
x19/S3 = 0x60023000	0x420103D0			
x20/S4 = 0x60043000	0x420103D4			
x21/S5 = 0x3fc81e74	0x420103D8			
x22/S6 = 0x00000000	0x420103DC			
x23/S7 = 0x00000000	0x420103E0			
x24/S8 = 0x00000000	0x420103E4			
x25/S9 = 0x3fc82e74	0x420103E8			
x26/S10 = 0x000003e8	0x420103EC			
x27/S11 = 0x00001000	0x420103F0			
x28/T3 = 0x00000001	0x420103F4			
x29/T4 = 0x3fc84e70	0x420103F8			
x30/T5 = 0x00000000	0x420103FC			
x31/T6 = 0x00000000	0x42010400			
PC/PC = 0x420103bc	0x42010404			
WATCH	0x42010408			
	0x4201040C			
	0x42010410			
	0x42010414			
	0x42010418			
	0x4201041C			
	0x42010420			
	0x42010424			
	0x42010428			
	0x4201042C			

Figure 21 - Exécution pas à pas



Assurez-vous que les adresses correspondent à la plage donnée par les logs du GoLViewer. Les labels n'existent pas dans la vue de désassemblage.

Vous pouvez ensuite parcourir le code et regarder les registres se mettre à jour.



Glossaire

ASM – Assembly [9](#), [18](#)

CPU – Central Processing Unit [12](#), [13](#), [18](#)

GoL – Game of Life [3](#), [6](#), [8](#)

HITL – Hardware In The Loop [2](#), [12](#)

ISA – Instruction Set Architecture [5](#)

LED – Light Emitting Diode [3](#)

RGB – Red Green Blue [9](#)

RISCV – 5-stages Reduced Instruction Set Computer architecture, open-sourced [4](#), [11](#), [12](#), [17](#)

UART – Universal Asynchronous Receiver Transmitter [11](#), [12](#), [13](#), [17](#), [18](#)



Bibliographie

- [1] Christophe Bianchi, François Corthay, et Silvan Zahno, « Comment Rédiger Un Rapport de Projet? ». 2021.